Projeto 2ºGQ: Comparação de Estruturas de Dados

Informações gerais

- **Equipes:** 2 ou 3 integrantes, com no máximo 8 equipes.
- **Valor:** 4,0 pontos.
- **Entrega:** até 22 de novembro, às 23h59, pelo Microsoft Teams.
- **Apresentações:** 23 e 26 de novembro.
- **Duração:** 10 a 20 minutos por equipe.

Um sistema de Cadastro Nacional de Estudantes reúne aproximadamente 10 milhões de registros históricos de alunos de diferentes instituições de ensino superior brasileiras. Cada registro contém uma matrícula única, nome completo e curso. Diante do grande volume de informações, é necessário avaliar diferentes estruturas de dados para identificar quais oferecem melhor desempenho nas operações de inserção e busca de estudantes por matrícula. O objetivo do projeto é implementar e comparar o desempenho de diferentes estruturas de dados, utilizando a linguagem C e uma base de dados contendo 10 milhões de registros.

1. Estruturas de dados

Cada equipe deverá implementar três estruturas:

1. Estrutura linear: escolher uma entre lista unicamente encadeada, lista duplamente encadeada ou lista circular.
2. Tabela hash com endereçamento aberto: implementar uma estratégia de sondagem (linear, quadrática ou dupla).
3. Tabela hash com encadeamento separado: utilizar listas encadeadas para tratar colisões.

As implementações devem utilizar alocação dinâmica de memória e ser organizadas em módulos, seguindo o conceito de Tipo Abstrato de Dados (TAD).

2. Funções hash

As equipes deverão implementar duas funções hash distintas:

1. Primeira função: utilizar o método da divisão ou da multiplicação, apresentados em sala.
2. Segunda função: pesquisar e implementar outro método de hashing, diferente do primeiro.

3. Experimentos e comparação

Cada equipe deverá comparar o desempenho das estruturas utilizando os mesmos registros e as mesmas matrículas nas operações de busca. As duas funções hash deverão ser testadas tanto com sondagem quanto com encadeamento separado, totalizando quatro configurações de tabelas hash, além da estrutura linear escolhida.

3.1. Preparação dos experimentos

Utilizar a base de dados disponibilizada, preservando a ordem dos registros, e realizar os experimentos com três volumes:

- 100 mil registros.
- 1 milhão de registros.
- 10 milhões de registros.

Para cada volume, inicializar as cinco implementações vazias e inserir os registros progressivamente, realizando medições após atingir 30%, 60% e 100% do volume considerado. As quatro configurações de tabelas hash deverão utilizar o mesmo tamanho de tabela para cada volume de dados, mantendo essa capacidade fixa durante o experimento.

3.2. Operações e métricas

Para cada volume de dados, realizar:

1. Inserção: medir separadamente o tempo necessário para inserir os registros nos intervalos de 0–30%, 30–60% e 60–100%.
2. Busca: após cada etapa de inserção, realizar 10.000 buscas por matrículas existentes, selecionadas aleatoriamente entre os registros já inseridos, e medir o tempo total.
3. Colisões: contabilizar, em cada intervalo de inserção, quantas vezes uma nova matrícula produz um índice hash inicial que já foi produzido por outra matrícula.
4. Fator de carga: calcular o fator de carga das tabelas hash após cada etapa de inserção.

O fator de carga (α) representa a relação entre a quantidade de elementos armazenados (n) e o tamanho da tabela hash (m):

$$\alpha = n / m$$

No endereçamento aberto, o fator de carga varia de 0 a 1. No encadeamento separado, pode ultrapassar 1, pois uma posição pode armazenar vários elementos.

3.3. Condições dos experimentos

Para garantir a comparação entre as implementações:

- Utilizar os mesmos registros, na mesma ordem de inserção.
- Selecionar aleatoriamente 10.000 matrículas entre os registros já inseridos em cada etapa, utilizando as mesmas matrículas e a mesma ordem de busca nas cinco implementações.
- Não incluir o tempo de leitura do arquivo nem o tempo de seleção das matrículas nas medições.
- Utilizar o mesmo tamanho de tabela hash nas quatro configurações, mantendo essa capacidade fixa durante cada experimento.
- A escolha do tamanho da tabela hash deverá ser justificada.

3.4. Comparação dos resultados

Cada equipe deverá apresentar tabelas com os tempos de inserção e busca das cinco implementações, para os três volumes de dados e os três níveis de preenchimento. Para as tabelas hash, deverão apresentar também a quantidade de colisões e o fator de carga em cada etapa.

Os resultados deverão ser analisados considerando:

- As diferenças de desempenho entre as estruturas.
- O comportamento dos tempos conforme aumenta a quantidade de registros.

- As diferenças entre as funções hash e as estratégias de tratamento de colisões.

Ao final, cada equipe deverá identificar quais implementações apresentaram os melhores resultados e justificar as diferenças observadas. Os experimentos completos deverão ser executados previamente e documentados no repositório.

4. Entrega e apresentação

Cada equipe deverá disponibilizar e submeter um link para um repositório público no GitHub contendo:

- Código-fonte organizado em módulos.
- Arquivo `README.md` com descrição das implementações, instruções de compilação e execução.
- Slides contendo os resultados dos experimentos, apresentados em tabelas ou gráficos, e as principais conclusões.

Todos os integrantes deverão participar da apresentação e estar preparados para explicar qualquer parte do código.

5. Avaliação

O projeto valerá 4,0 pontos, distribuídos da seguinte forma:

- Implementação (1,0 ponto): funcionamento das estruturas, organização do código e realização dos experimentos.
- Apresentação (3,0 pontos): domínio das implementações, apresentação dos resultados, demonstração do código e participação dos integrantes.